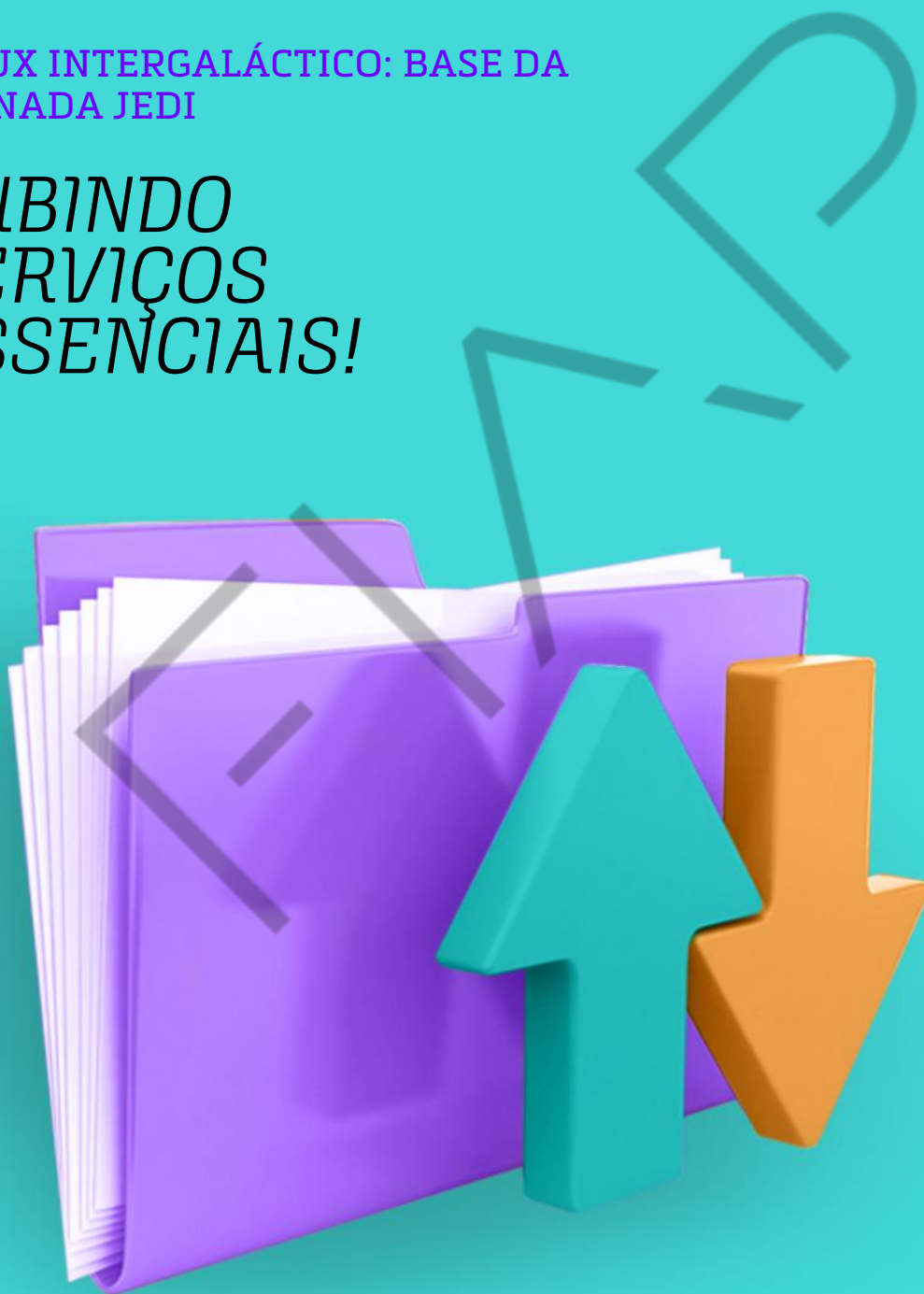


LINUX INTERGALÁCTICO: BASE DA
JORNADA JEDI

*SUBINDO
SERVIÇOS
ESSENCIAIS!*



7

LISTA DE FIGURAS

Figura 1 – Instalação do serviço vsftpd usando o comando apt.....	7
Figura 2 – Inicialização do vsftpd usando o comando systemctl.....	7
Figura 3 – Menu dispositivos, configuração de rede no Oracle VirtualBox	8
Figura 4 – Placa definida em modo bridge no Oracle VirtualBox	9
Figura 5 – Teste de conectividade do serviço vsftpd.....	9
Figura 6 – Um breve questionário do comando openssl	12
Figura 7 – Inicialização do vsftpd usando o comando systemctl.....	13
Figura 8 – Site do WinSCP.....	14
Figura 9 – Configuração SFTP do cliente WinSCP	14
Figura 10 – Informações da chave sendo exibidas	15
Figura 11 – Serviço SFTP funcionando no WinSCP	15
Figura 12 - Instalação do serviço openssh-server usando o comando apt	17
Figura 13 – Página web do cliente ssh Putty	19
Figura 14 – Teste de conectividade do serviço openssh-server usando o cliente Putty	19
Figura 15 – Apresentação da chave pública no cliente Putty.....	20
Figura 16 – Teste de conectividade do Putty com sucesso	21
Figura 17 – Configurando o WinSCP para conexão SCP	21
Figura 18 – Apresentação da chave WinSCP	22
Figura 19 – Apresentação da chave WinSCP	23
Figura 20 – Realizando o acesso SSH por meio da porta 2222.....	25
Figura 21 – Geração de chave usando o Puttygen	26
Figura 22 – Chave gerada pelo Puttygen.....	26
Figura 23 – Copiando a chave pública gerada no Puttygen.....	27
Figura 24 – Adicionando a chave pública no arquivo /home/ubuntu/.ssh/authorized_keys.....	27
Figura 25 – Salvando o arquivo de chave privada do Puttygen	28
Figura 26 – Acesso proibido sem a chave ao usar o Putty	29
Figura 27 – Acesso proibido sem a chave ao usar o Putty	30
Figura 28 – Informando a chave privada arquivo .ppk no Putty	30
Figura 29 – Acesso com arquivo de chave privada usando o Putty	31
Figura 30 – Instalar o serviço knockd.....	32
Figura 31 – Verificando e testando acesso na porta 22 do ssh.....	35
Figura 32 – <i>Port-knocking</i> realizado com sucesso	36

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 – Configuração de IPv6 e desabilitação do login anônimo	10
Código-fonte 2 – Configuração de IPv6 e desabilitação do login anônimo	11
Código-fonte 3 – Configuração de IPv6 e desabilitação do login anônimo	11
Código-fonte 4 – Descomente para restringir os usuários aos seus diretórios locais	11
Código-fonte 5 – Configuração genética para protocolo SFTP no vsFTPD	12
Código-fonte 6 – Configuração para protocolo SFTP no vsFTPD	13
Código-fonte 7 – Descomente para restringir os usuários aos seus diretórios locais	24
Código-fonte 8 – Descomente para restringir os usuários aos seus diretórios locais	24
Código-fonte 9 – Arquivo de configuração /etc/ssh/sshd_config	28
Código-fonte 10 – Arquivo de configuração /etc/ssh/sshd_config com PasswordAuthentication em no	28
Código-fonte 11 – O arquivo /etc/firewall.sh	33
Código-fonte 12 – O arquivo /etc/knockd.conf com uma ligeira alteração	34
Código-fonte 13 – O arquivo /etc/default/knock alterado	34
Código-fonte 14 – Shell script que realiza o <i>port-knocking</i>	35

LISTA DE COMANDOS DE PROMPT DO SISTEMA OPERACIONAL

Comando de prompt 1 – Instalação do serviço vsftpd usando o comando apt	6
Comando de prompt 2 – Inicialização do vsftpd usando o comando systemctl	7
Comando de prompt 3 – Criando uma chave com o comando openssl	12
Comando de prompt 4 – Reiniciando o vsftpd usando o comando systemctl	13
Comando de prompt 5 – Instalação do serviço openssh-server usando o comando apt	16
Comando de prompt 6 – Inicialização do openssh-server usando o comando systemctl	17
Comando de prompt 7 – Inicialização do openssh-server usando o comando systemctl	24
Comando de prompt 8 – Definindo as permissões de .ssh corretamente	29
Comando de prompt 9 – Reinicialização do openssh-server usando o comando systemctl	29

SUMÁRIO

SUBINDO SERVIÇOS ESSENCIAIS!	6
1. INTRODUÇÃO	6
2 SERVIÇO DE FTP	6
2.1 Instalando o serviço vsftpd	6
2.2 Testando o serviço vsftpd.....	8
2.3 Configurações do serviço vsftpd	10
3 SERVIÇO DE SSH.....	16
3.1 Instalando o serviço OPENSSH SERVER	16
3.2 Testando o serviço OPENSSH-SERVER.....	18
3.3 Configurando o serviço OPENSSH-SERVER	23
3.4 Realizando acesso ssh com uma chave no lugar da senha.....	25
3.5 <i>Port-knocking</i>	31
REFERÊNCIA	37

SUBINDO SERVIÇOS ESSENCIAIS!

1. INTRODUÇÃO

Alguns serviços no Linux são essenciais, e diria que nada é mais essencial do que trocar arquivos e executar comandos em um sistema operacional. A seguir, vamos ver como implementar os serviços de FTP e SSH da forma mais segura possível.

2 SERVIÇO DE FTP

O serviço de FTP é um dos mais tradicionais e básicos de um servidor, e um dos alicerces da Internet em seus primórdios. Sem um serviço projetado especificamente para a troca de arquivos (a sigla significa **File Transfer Protocol** ou protocolo de transferência de arquivos) é seguro dizer que a rede mundial de computadores jamais teria prosperado como prosperou.

A seguir, veremos como instalar, testar e configurar corretamente o vsftpd, um dos servidores de FTP mais utilizados atualmente.

2.1 Instalando o serviço vsftpd

Para instalar o vsftpd em um Linux do tipo Debian/Ubuntu, utilize o comando apt conforme exibido a seguir. Em distribuições baseadas em Red Hat, o comando equivalente é utilizando o yum:

```
$ sudo apt install vsftpd
```

Comando de prompt 1 – Instalação do serviço vsftpd usando o comando apt
Fonte: Elaborado pelo autor (2021)

Seja o apt ou o yum, o processo é o mesmo: os repositórios presentes em `/etc/apt/sources.list` e `/etc/apt/sources.list.d` (Debian/Ubuntu, mantenha as referências sempre atualizadas com o comando `sudo apt update`) são consultados, e o pacote de instalação e suas dependências são baixados e instalados automaticamente, como pode ser visto a seguir.

```
ubuntu@ubuntu-VirtualBox:~$ sudo apt install vsftpd
Lendo listas de pacotes... Pronto
Construindo árvore de dependências
Lendo informação de estado... Pronto
Os seguintes pacotes foram instalados automaticamente e já não são necessários:
  ncurses-term openssh-sftp-server ssh-import-id
Utilize 'sudo apt autoremove' para os remover.
Os NOVOS pacotes a seguir serão instalados:
  vsftpd
0 pacotes atualizados, 1 pacotes novos instalados, 0 a serem removidos e 132 não a
tualizados.
É preciso baixar 115 kB de arquivos.
Depois desta operação, 338 kB adicionais de espaço em disco serão usados.
Obter:1 http://br.archive.ubuntu.com/ubuntu groovy/main amd64 vsftpd amd64 3.0.3-1
2 [115 kB]
Baixados 115 kB em 3s (40,5 kB/s)
Pré-configurando pacotes ...
A seleccionar pacote anteriormente não seleccionado vsftpd.
(Lendo banco de dados ... 200235 ficheiros e directórios actualmente instalados.)
A preparar para desempacotar .../vsftpd_3.0.3-12_amd64.deb ...
A descompactar vsftpd (3.0.3-12) ...
Configurando vsftpd (3.0.3-12) ...
Created symlink /etc/systemd/system/multi-user.target.wants/vsftpd.service → /lib/
systemd/system/vsftpd.service.
vsftpd.conf:1: Line references path below legacy directory /var/run/, updating /va
r/run/vsftpd/empty → /run/vsftpd/empty; please update the tmpfiles.d/ drop-in file
accordingly.
A processar 'triggers' para man-db (2.9.3-2) ...
A processar 'triggers' para systemd (246.6-1ubuntu1) ...
ubuntu@ubuntu-VirtualBox:~$
```

Figura 1 – Instalação do serviço vsftpd usando o comando apt

Fonte: Elaborado pelo autor (2021)

Uma vez instalado, vamos colocar o serviço em funcionamento para vê-lo em ação. O primeiro comando habilita o serviço para inicializar automaticamente com o sistema (ou seja, quando o Linux for iniciado) enquanto o segundo comando sobe o serviço imediatamente.

```
# Habilitando o serviço vsftpd para subir na inicialização do
# sistema
$ sudo systemctl enable vsftpd
# Iniciando o serviço vsftpd agora
$ sudo systemctl start vsftpd
```

Comando de prompt 2 – Inicialização do vsftpd usando o comando systemctl

Fonte: Elaborado pelo autor (2021)

A imagem a seguir mostra os comandos sendo executados, embora em uma execução de sucesso não haja muito a se ver.

```
ubuntu@ubuntu-VirtualBox:~$ sudo systemctl enable vsftpd
Synchronizing state of vsftpd.service with SysV service script with /lib/systemd/s
ysv-install.
Executing: /lib/systemd/systemd-sysv-install enable vsftpd
ubuntu@ubuntu-VirtualBox:~$ sudo systemctl start vsftpd
ubuntu@ubuntu-VirtualBox:~$
```

Figura 2 – Inicialização do vsftpd usando o comando systemctl

Fonte: Elaborado pelo autor (2021)

2.2 Testando o serviço vsftpd

Agora que o serviço vsftpd está instalado e aparentemente está funcionando, vamos testar e descobrir se funciona mesmo? Claro que para o teste ficar mais interessante, convém testar o serviço a partir de uma máquina diferente.

No exemplo em questão, a máquina Linux (um Ubuntu) representa o servidor FTP e está virtualizado em um Oracle VirtualBox, e a máquina cliente é um Microsoft Windows e se trata da máquina real, o hospedeiro. Para tornar as coisas ainda mais interessantes, a configuração de rede do Oracle VirtualBox está com placa em modo bridge, assim, o sistema convidado (o Linux virtualizado) consegue acessar a Internet, mas possui seu próximo endereço IP na rede local, um endereço diferente da máquina hospedeira. Desta forma, uma pode acessar a outra normalmente. Nas imagens a seguir, podemos ver a configuração no Oracle VirtualBox.

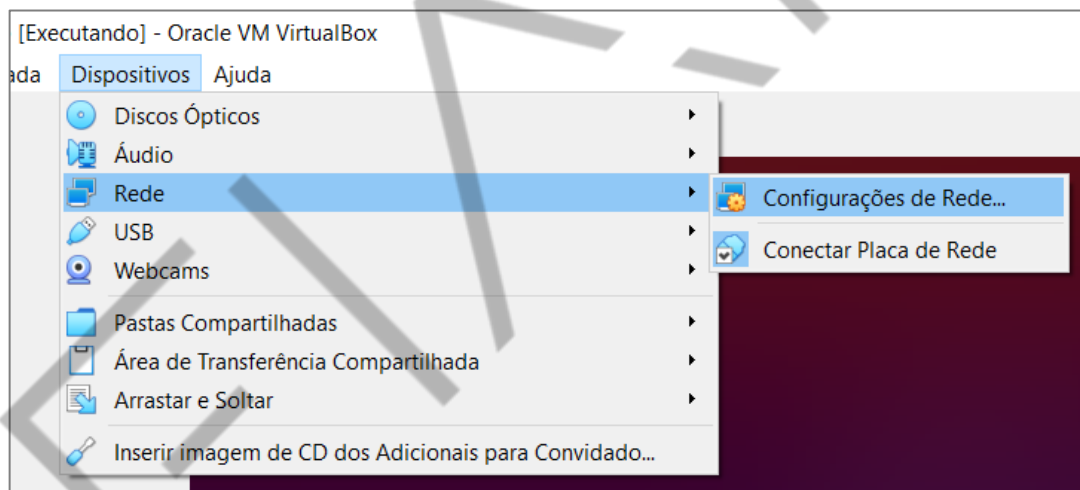


Figura 3 – Menu dispositivos, configuração de rede no Oracle VirtualBox
Fonte: Elaborado pelo autor (2021)

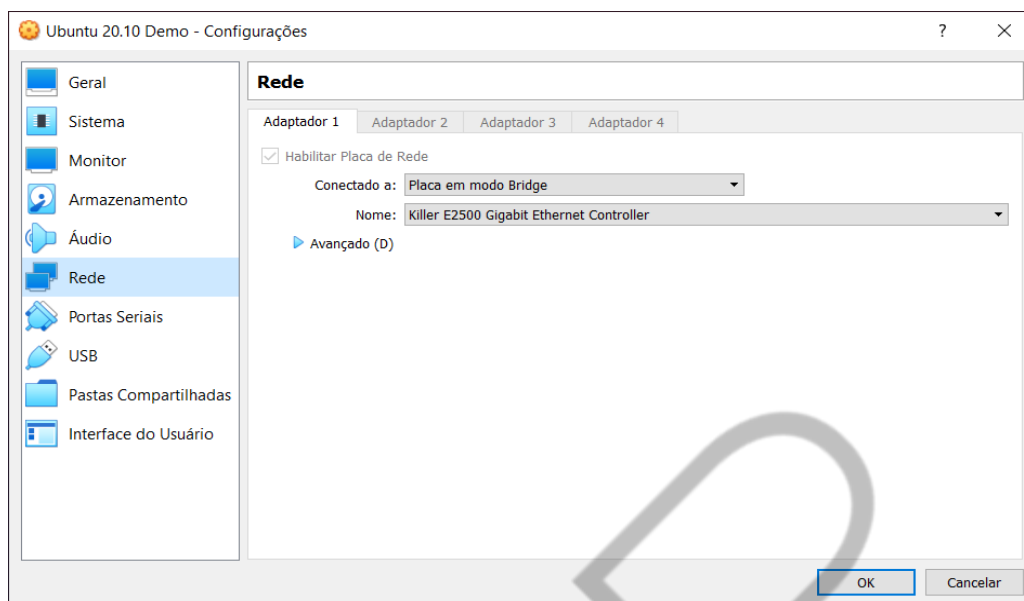


Figura 4 – Placa definida em modo bridge no Oracle VirtualBox
Fonte: Elaborado pelo autor (2021)

Para acessar o servidor FTP, é necessário saber o endereço IP (neste caso, foi utilizado o IPv4) do servidor Linux, e para isso basta usar o comando **ip addr show**. Com o endereço IPv4 em mãos, o terminal de comandos do Windows foi aberto e o cliente de ftp foi chamado. Repare que o vsFTPD atende normalmente, informando sua versão (neste caso, 3.0.3) e solicitando usuário e senha do usuário que, uma vez fornecida corretamente, permite o uso do protocolo.

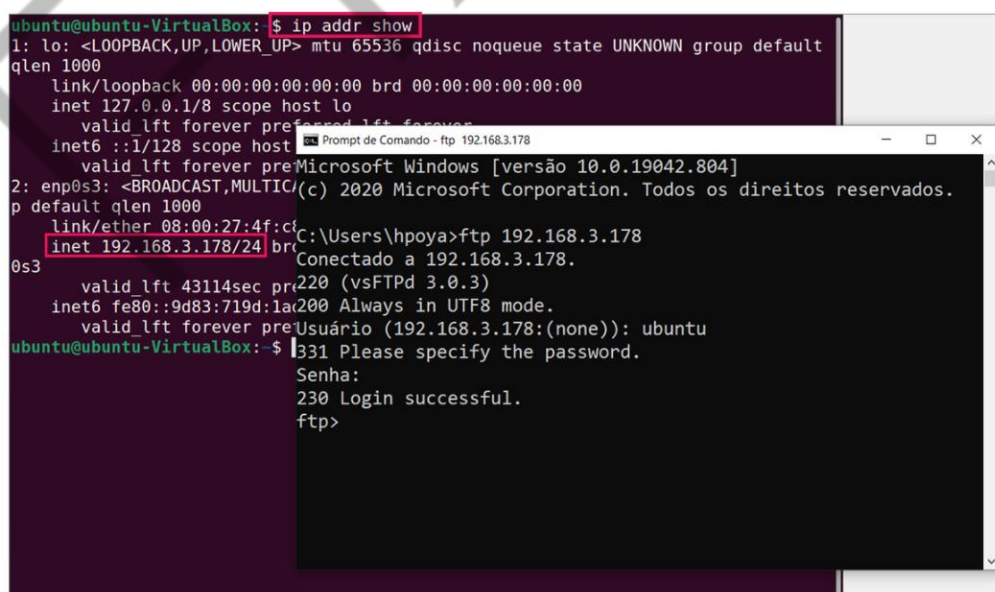


Figura 5 – Teste de conectividade do serviço vsftpd
Fonte: Elaborado pelo autor (2021)

2.3 Configurações do serviço vsftpd

Bem, agora que já sabemos instalar e que o serviço funciona, vamos realizar algumas configurações no serviço de forma a torná-lo mais seguro. Na verdade, não deveríamos ter levantado o serviço em sua configuração original: embora as distribuições optem por configurações iniciais razoavelmente seguras, elas sempre são permissivas demais em prol da funcionalidade do serviço, assim sendo, cabe sempre ao responsável pela infraestrutura configurar o serviço de forma que este esteja seguro e atenda às necessidades da organização.

O arquivo de configuração geralmente se encontra em `/etc/vsftpd.conf`. Não cabe aqui explorar todas as configurações do serviço, portanto, vamos dar uma passada nas mais interessantes. O arquivo de configuração pode ser aberto em um editor de textos, como o `vi` ou o `nano`.

No início do arquivo, é possível ver a configuração para endereços IPv6 ativa, além do login anônimo estar desabilitado por padrão. Repare que ambas as linhas de configuração não possuem o símbolo “#” em seu início (como várias outras linhas), logo, as configurações estão ativas.

```
# This directive enables listening on IPv6 sockets. By
# default, listening
# on the IPv6 "any" address (::) will accept connections
# from both IPv6
# and IPv4 clients. It is not necessary to listen on *both*
# IPv4 and IPv6
# sockets. If you want that (perhaps because you want to
# listen on specific
# addresses) then you must run two copies of vsftpd with
# two configuration
# files.
listen_ipv6=YES
#
# Allow anonymous FTP? (Disabled by default).
anonymous_enable=NO
```

Código-fonte 1 – Configuração de IPv6 e desabilitação do login anônimo
Fonte: Elaborado pelo autor (2021)

Mais adiante, temos a `ftpd_banner`, que se encontra comentada. Esta configuração permite personalizar a mensagem de boas-vindas no serviço de FTP, quando alguém o acessa.

```
# You may fully customise the login banner string:  
#ftpd_banner=Welcome to blah FTP service.
```

Código-fonte 2 – Configuração de IPv6 e desabilitação do login anônimo
Fonte: Elaborado pelo autor (2021)

Vamos descomentar esta configuração retirando o “#” do início da linha e trocar a mensagem por algo mais sugestivo.

```
# You may fully customise the login banner string:  
ftpd_banner=Bem-vindo ao FTP do Poyatos
```

Código-fonte 3 – Configuração de IPv6 e desabilitação do login anônimo
Fonte: Elaborado pelo autor (2021)

Outra configuração interessante para se descomentar é **chroot_local_user**. Caso opte por ativá-la, ela restringe os usuários aos seus diretórios home, impedindo-os de navegar em outros diretórios, ainda que haja permissão para isso. Como é possível habilitar uma lista de exceção, vale a pena pesquisar mais sobre a funcionalidade (e testá-la) antes de colocá-la em prática, em produção.

```
# You may restrict local users to their home directories.  
See the FAQ for  
# the possible risks in this before using chroot_local_user  
or  
# chroot_list_enable below.  
#chroot_local_user=YES
```

Código-fonte 4 – Descomente para restringir os usuários aos seus diretórios locais
Fonte: Elaborado pelo autor (2021)

Por fim, uma possibilidade interessante: habilitar criptografia no serviço FTP! Em sua forma padrão, toda a troca de informações do protocolo (incluindo o processo de autenticação, informando o login e senha) é feito de forma aberta e pode ser facilmente interceptado (Você duvida? Pois retorne ao capítulo 16 da Fase 2, intitulado “O protocolo TCP/IP sob ataque!” que a gente mostra isso na prática, no protocolo FTP, inclusive).

```
# This option specifies the location of the RSA certificate
to use for SSL
# encrypted connections.
rsa_cert_file=/etc/ssl/certs/ssl-cert-snakeoil.pem
rsa_private_key_file=/etc/ssl/private/ssl-cert-snakeoil.key
ssl_enable=NO
```

Código-fonte 5 – Configuração genética para protocolo SFTP no vsFTPD

Fonte: Elaborado pelo autor (2021)

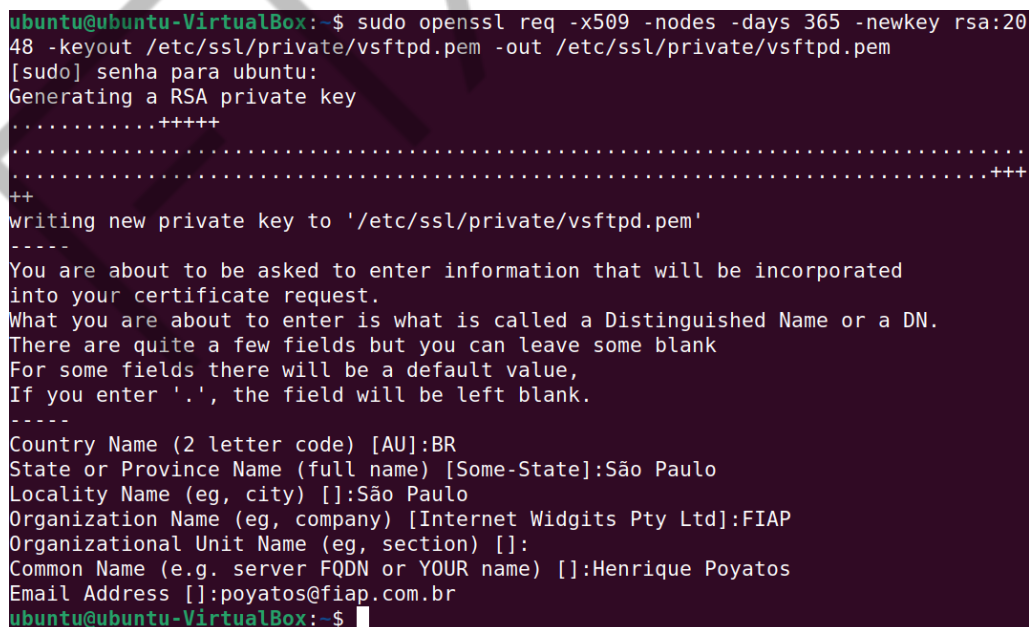
Mas antes, vamos criar a chave que será utilizada no protocolo seguro, pois a mencionada no arquivo de configuração é apenas um exemplo. O comando **openssl** a seguir possui alguns parâmetros para criar uma chave RSA de 2048 bits com a validade de 365 dias.

```
$ sudo openssl req -x509 -nodes -days 365 -newkey rsa:2048 -
keyout /etc/ssl/private/vsftpd.pem -out
/etc/ssl/private/vsftpd.pem
```

Comando de prompt 3 – Criando uma chave com o comando openssl

Fonte: Elaborado pelo autor (2021)

Conforme pode ser visto na imagem a seguir, algumas informações devem ser preenchidas na chave, então basta responder as perguntas que surgem a seguir.



```
ubuntu@ubuntu-VirtualBox:~$ sudo openssl req -x509 -nodes -days 365 -newkey rsa:20
48 -keyout /etc/ssl/private/vsftpd.pem -out /etc/ssl/private/vsftpd.pem
[sudo] senha para ubuntu:
Generating a RSA private key
.....+++++
.....+++++
++
writing new private key to '/etc/ssl/private/vsftpd.pem'
-----
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
-----
Country Name (2 letter code) [AU]:BR
State or Province Name (full name) [Some-State]:São Paulo
Locality Name (eg, city) []:São Paulo
Organization Name (eg, company) [Internet Widgits Pty Ltd]:FIAP
Organizational Unit Name (eg, section) []:
Common Name (e.g. server FQDN or YOUR name) []:Henrique Poyatos
Email Address []:poyatos@fiap.com.br
ubuntu@ubuntu-VirtualBox:~$
```

Figura 6 – Um breve questionário do comando openssl

Fonte: Elaborado pelo autor (2021)

Com a chave devidamente criada, as três linhas do arquivo de configuração agora podem ser devidamente configuradas.

```
# This option specifies the location of the RSA certificate
to use for SSL
# encrypted connections.
rsa_cert_file=/etc/ssl/private/vsftpd.pem
rsa_private_key_file=/etc/ssl/private/vsftpd.pem
ssl_enable=YES
```

Código-fonte 6 – Configuração para protocolo SFTP no vsFTPd
Fonte: Elaborado pelo autor (2021)

Salve e saia do arquivo de configuração e, para que as mudanças tenham efeito, é necessário reiniciar o serviço.

```
$ sudo systemctl restart vsftpd
```

Comando de prompt 4 – Reiniciando o vsftpd usando o comando systemctl
Fonte: Elaborado pelo autor (2021)

Observe que, ao tentar acessar o serviço de FTP pela segunda vez, embora tenha sido bem recebido pelo banner personalizado, ao digitar o usuário, ele informa que a seção deve ser criptografada agora, de forma que o cliente Windows não está disposto a fazer.

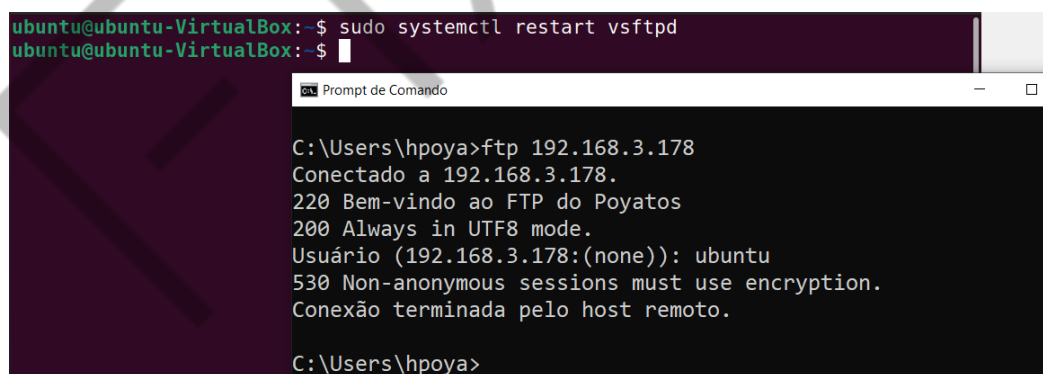


Figura 7 – Inicialização do vsftpd usando o comando systemctl
Fonte: Elaborado pelo autor (2021)

Para demonstrar a conexão SFTP, vamos utilizar aqui o WinSCP que pode ser baixado no endereço <https://winscp.net/eng/download.php>. Ele será muito útil na demonstração do serviço de SSH também, é muito útil e pode ser instalado gratuitamente no Windows. Caso esteja em um MacOS ou Linux, fique à vontade para buscar um aplicativo equivalente para esta parte da demonstração.

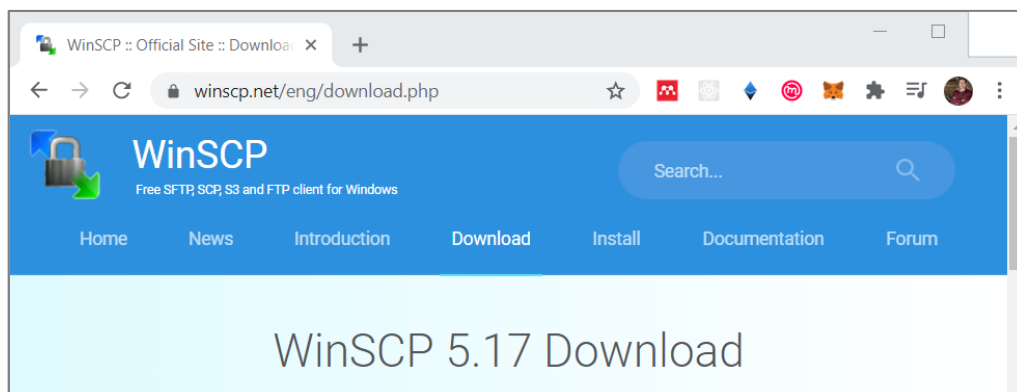


Figura 8 – Site do WinSCP
Fonte: Elaborado pelo autor (2021)

Em “Novo Site”, a configuração deve ser “FTP” e criptografia do tipo explícita “TLS/SSL”.

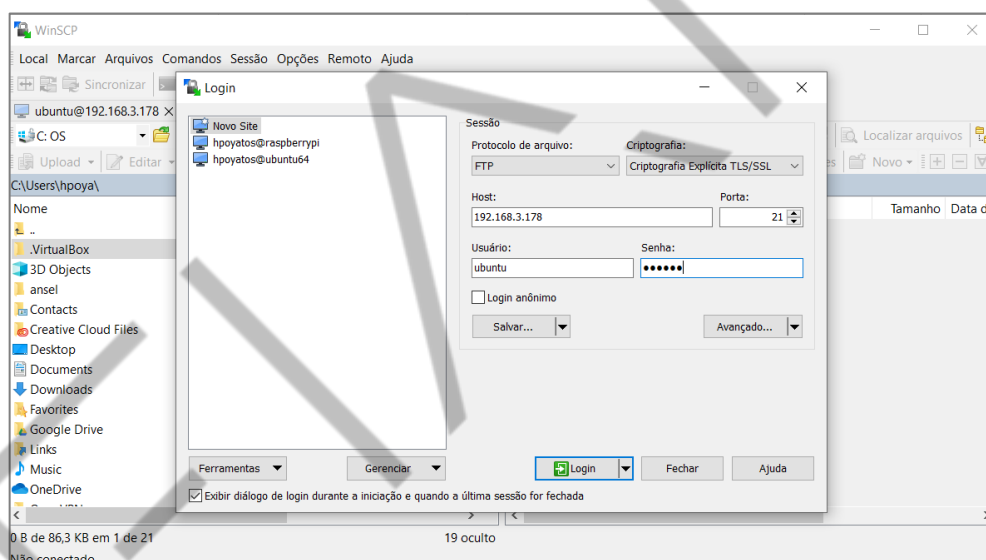


Figura 9 – Configuração SFTP do cliente WinSCP
Fonte: Elaborado pelo autor (2021)

Ao clicar no botão “Login”, repare que as informações da chave que acabamos de preencher são exibidas aqui. Embora ele diga que não há garantias que se trata do servidor que diz ser, bem, acho que podemos afirmar que é nosso servidor atendendo por aqui.



Figura 10 – Informações da chave sendo exibidas
Fonte: Elaborado pelo autor (2021)

Por fim, temos um ótimo cliente para SFTP acessando nosso servidor Linux de uma forma mais segura, pronto para receber nossos arquivos.

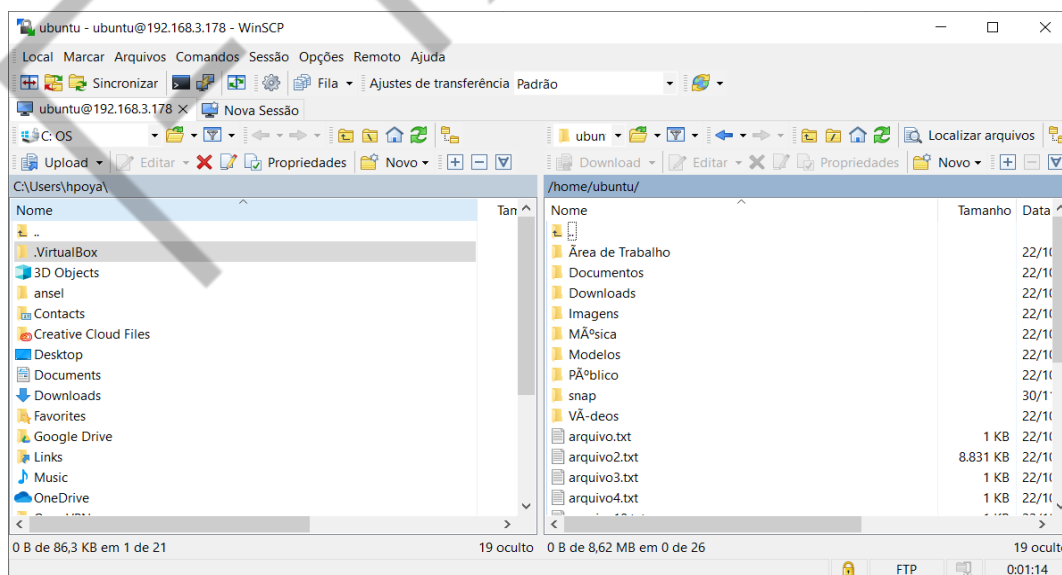


Figura 11 – Serviço SFTP funcionando no WinSCP
Fonte: Elaborado pelo autor (2021)

Claro que cabe aqui mais recursos de segurança e um deles é a configuração de *firewall*, que será abordada com mais detalhes no capítulo 7 desta fase.

3 SERVIÇO DE SSH

Outro serviço indispensável é o de Secure Shell (SSH) ou o terminal de comandos *shell* seguro – trata-se de uma evolução do antigo serviço de telnet que, no passado, permitia a obtenção de acesso remoto a uma máquina Unix por meio de uma conexão discada. Outro fator importante é que o serviço de telnet não protegia sua conexão com criptografia (não era necessário), sendo indispensável atualmente, afinal, basta um simples *sniffer* na rede para roubar das credenciais, conhecer todos os comandos e dados trafegados em uma conexão remota não criptografada.

O serviço amplamente utilizado em distribuições Unix e Linux mundo afora é o OpenSSH, mantido pela comunidade do OpenBSD, considerado um dos sistemas operacionais mais seguros do mundo. Sua instalação e configuração são abordadas a seguir.

3.1 Instalando o serviço OPENSSH SERVER

A instalação pode ser realizada de forma simples em um sistema operacional Linux do tipo Debian/Ubuntu por meio do comando `apt`. Para sistemas Red Hat e derivados, basta adequar o comando usando o `yum`.

```
$ sudo apt install openssh-server
```

Comando de prompt 5 – Instalação do serviço openssh-server usando o comando `apt`

Fonte: Elaborado pelo autor (2019)

O procedimento é o mesmo: o repositório informado em `/etc/apt/sources.list` e `/etc/apt/sources.list.d` é consultado e o pacote de instalação do `openssh-server` (e suas eventuais dependências) são baixadas automaticamente da Internet, desde que o Linux em questão possua conectividade naquele momento. Permissão dada para a instalação, os pacotes são instalados nos diretórios padrão, localizados na pasta `/usr`.


```
ubuntu@ubuntu-VirtualBox:~$ sudo apt install openssh-server
Lendo listas de pacotes... Pronto
Construindo árvore de dependências
Lendo informação de estado... Pronto
Pacotes sugeridos:
  molly-guard monkeysphere ssh-askpass
Os NOVOS pacotes a seguir serão instalados:
  openssh-server
0 pacotes atualizados, 1 pacotes novos instalados, 0 a serem removidos e 132 não a
tualizados.
É preciso baixar 378 kB de arquivos.
Depois desta operação, 1.526 kB adicionais de espaço em disco serão usados.
Obter:1 http://br.archive.ubuntu.com/ubuntu groovy/main amd64 openssh-server amd64
1:8.3p1-1 [378 kB]
Baixados 378 kB em 10s (36,6 kB/s)
Pré-configurando pacotes ...
A seleccionar pacote anteriormente não seleccionado openssh-server.
(Lendo banco de dados ... 200292 ficheiros e directórios actualmente instalados.)
A preparar para desempacotar .../openssh-server_1%3a8.3p1-1_amd64.deb ...
A descompactar openssh-server (1:8.3p1-1) ...
Configurando openssh-server (1:8.3p1-1) ...
rescue-ssh.target is a disabled or a static unit, not starting it.
A processar 'triggers' para systemd (246.6-1ubuntu1) ...
A processar 'triggers' para man-db (2.9.3-2) ...
A processar 'triggers' para ufw (0.36-7) ...
ubuntu@ubuntu-VirtualBox:~$
```

Figura 12 - Instalação do serviço openssh-server usando o comando apt

Fonte: Elaborado pelo autor (2021)

Uma vez instalado, os comandos a seguir permitirão que o servidor SSH seja inicializado automaticamente com o sistema operacional e, também, seja executado imediatamente.

```
# Habilitando o serviço openssh-server para subir na
# inicialização do sistema
$ sudo systemctl enable ssh
# Iniciando o serviço openssh-server agora
$ sudo systemctl start ssh
```

Comando de prompt 6 – Inicialização do openssh-server usando o comando systemctl

Fonte: Elaborado pelo autor (2021)

Muito bem, aparentemente o serviço está funcionando!

3.2 Testando o serviço OPENSSSH-SERVER

O servidor SSH, à princípio, está operacional, mas vamos ter certeza disso, façamos agora um teste de conectividade. Para tornar as coisas mais interessantes, faremos o teste usando duas máquinas, sendo uma máquina real (no caso aqui um Windows 10 se comportando como máquina cliente) e uma máquina virtualizada (o Linux em um Oracle VirtualBox, se comportando como servidor). Assim como mencionado no serviço anterior (FTP), a configuração de rede está em modo bridge, na qual cada uma das máquinas possui seu próprio endereçamento IP em uma rede interna. Contudo, a máquina real/hospedeira se comporta como um gateway e provê acesso a rede externa (Internet) para a máquina virtualizada, ou seja, o melhor de dois mundos.

Para acessar o serviço SSH a partir do cliente Windows, um dos clientes mais utilizados é o Putty, que pode ser baixado em:

<<https://www.chiark.greenend.org.uk/~sgtatham/putty/latest.html>>

Caso você esteja em um MacOS ou Linux, esta etapa de download é absolutamente desnecessária, já que o cliente ssh do terminal é mais do que necessário para a tarefa. Ainda bem!

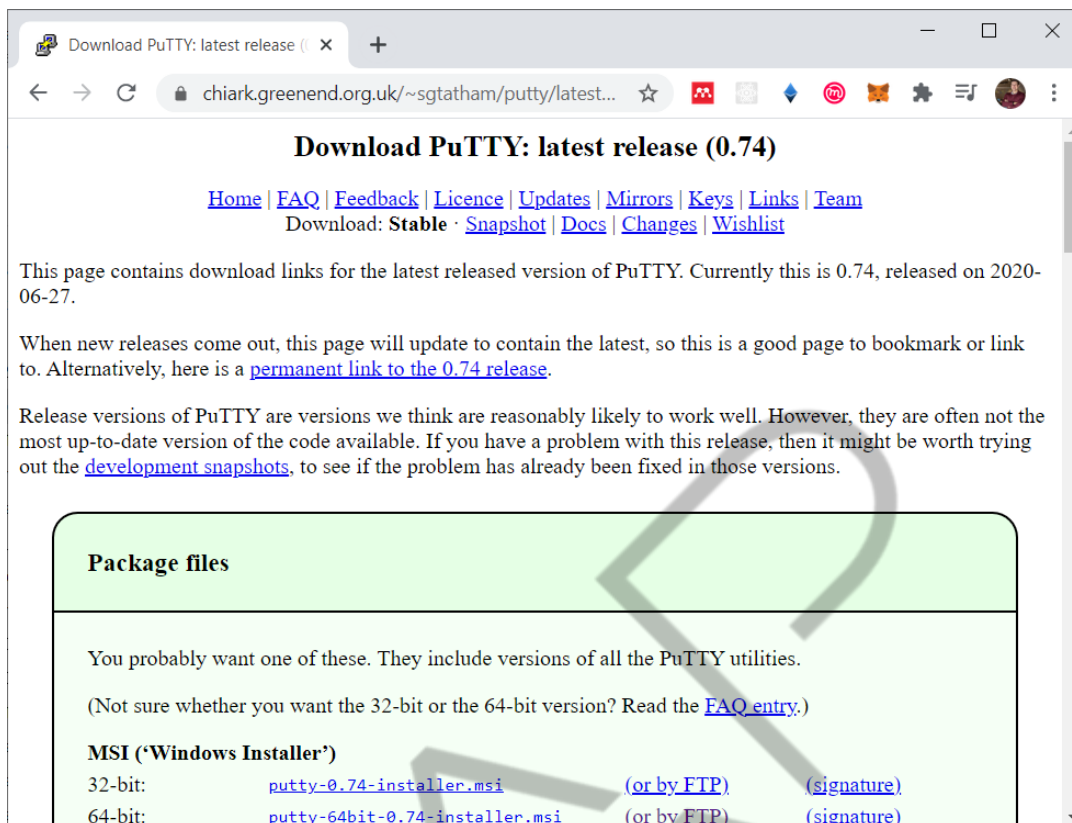


Figura 13 – Página web do cliente ssh Putty
Fonte: Elaborado pelo autor (2021)

Mais uma vez, utiliza-se o comando `ip addr show` para saber qual é o IPv4 exato do servidor Linux, e este por sua vez é informado no Putty (em seu equipamento o IPv4 será diferente, logo, informe sua própria numeração!).

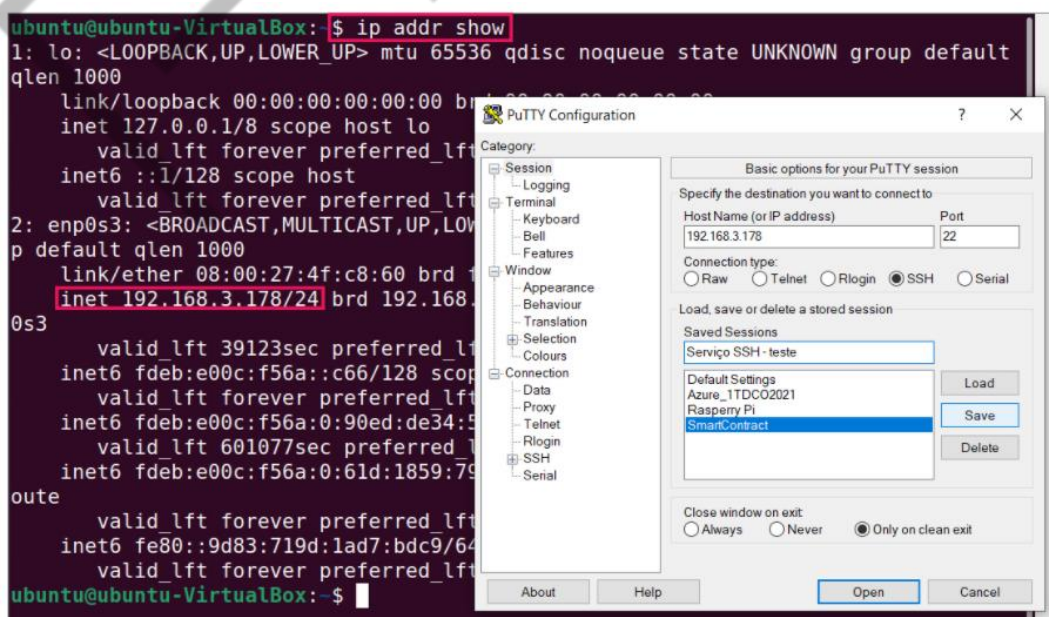


Figura 14 – Teste de conectividade do serviço openssh-server usando o cliente Putty
Fonte: Elaborado pelo autor (2021)

Ao clicar no botão “Open”, a chave pública do servidor é fornecida e informada pela primeira vez. Confiarei nesta chave com o botão de “Sim”.

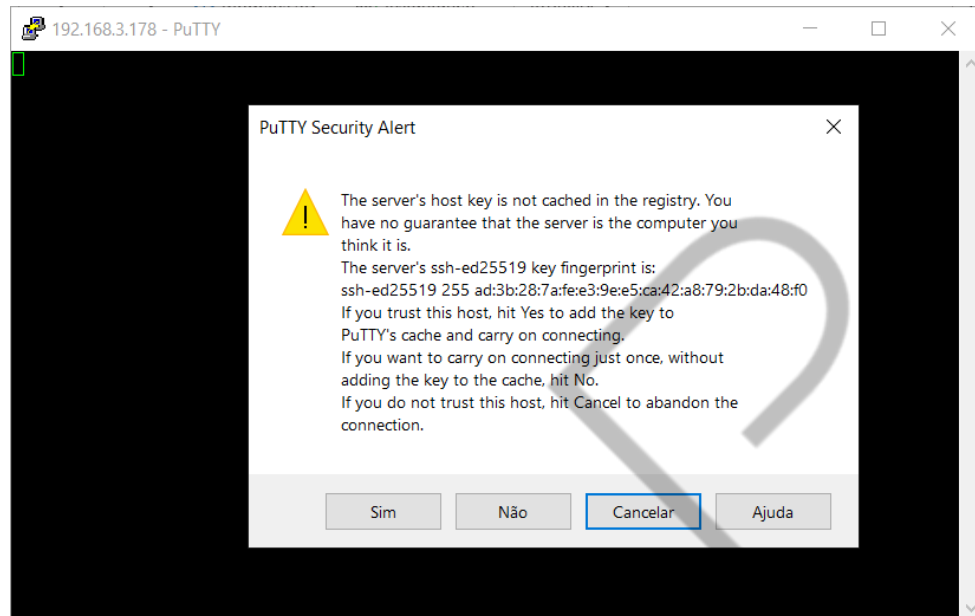
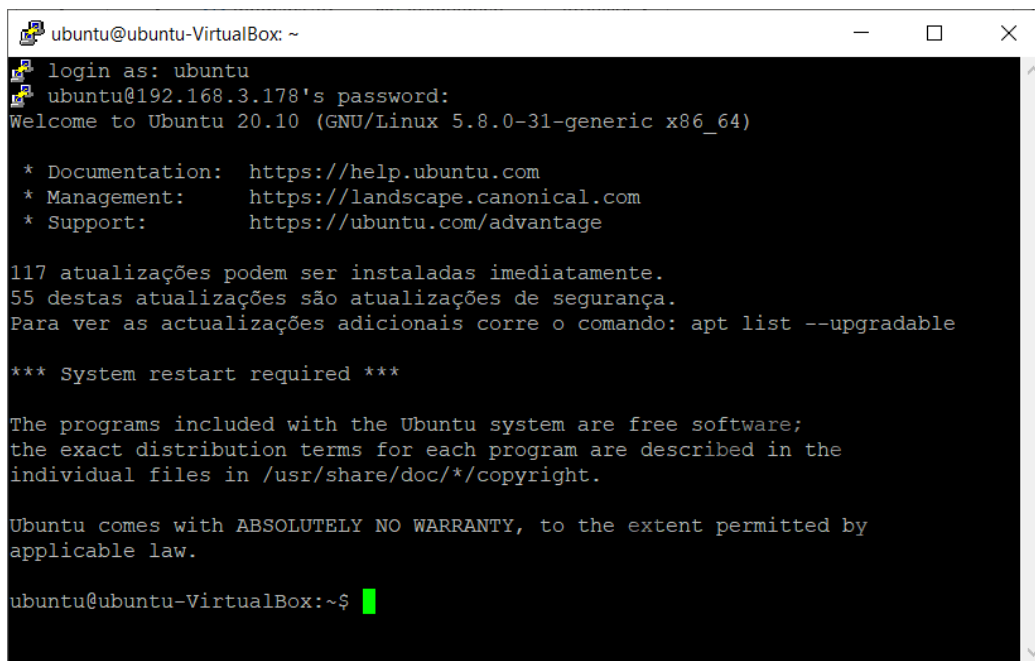


Figura 15 – Apresentação da chave pública no cliente Putty
Fonte: Elaborado pelo autor (2021)

O usuário e senha são solicitados pelo serviço de SSH. Ao informá-los corretamente, uma mensagem de boas-vindas surge, e o prompt padrão do meu usuário (no caso, o Bash, *Bourne Again Shell*) é apresentado.

A conexão foi um sucesso e, com o serviço de SSH, pode utilizar o sistema operacional exatamente da mesma forma que se você estivesse sentado fisicamente na presença da máquina.



```
ubuntu@ubuntu-VirtualBox: ~  
login as: ubuntu  
ubuntu@192.168.3.178's password:  
Welcome to Ubuntu 20.10 (GNU/Linux 5.8.0-31-generic x86_64)  
  
* Documentation:  https://help.ubuntu.com  
* Management:    https://landscape.canonical.com  
* Support:        https://ubuntu.com/advantage  
  
117 atualizações podem ser instaladas imediatamente.  
55 destas atualizações são atualizações de segurança.  
Para ver as actualizações adicionais corre o comando: apt list --upgradable  
  
*** System restart required ***  
  
The programs included with the Ubuntu system are free software;  
the exact distribution terms for each program are described in the  
individual files in /usr/share/doc/*/copyright.  
  
Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by  
applicable law.  
  
ubuntu@ubuntu-VirtualBox:~$
```

Figura 16 – Teste de conectividade do Putty com sucesso

Fonte: Elaborado pelo autor (2021)

É importante observar também que o serviço de SSH permite a cópia de arquivos de forma segura (SCP ou **Secure Copy**) e, para tal, posso utilizar o WinSCP, que foi instalado para testes no serviço anterior (FTP). O protocolo de conexão, contudo, deixa de ser SFTP e passa a ser SCP, nesse caso, apontando para a porta padrão de serviço SSH (porta 22).

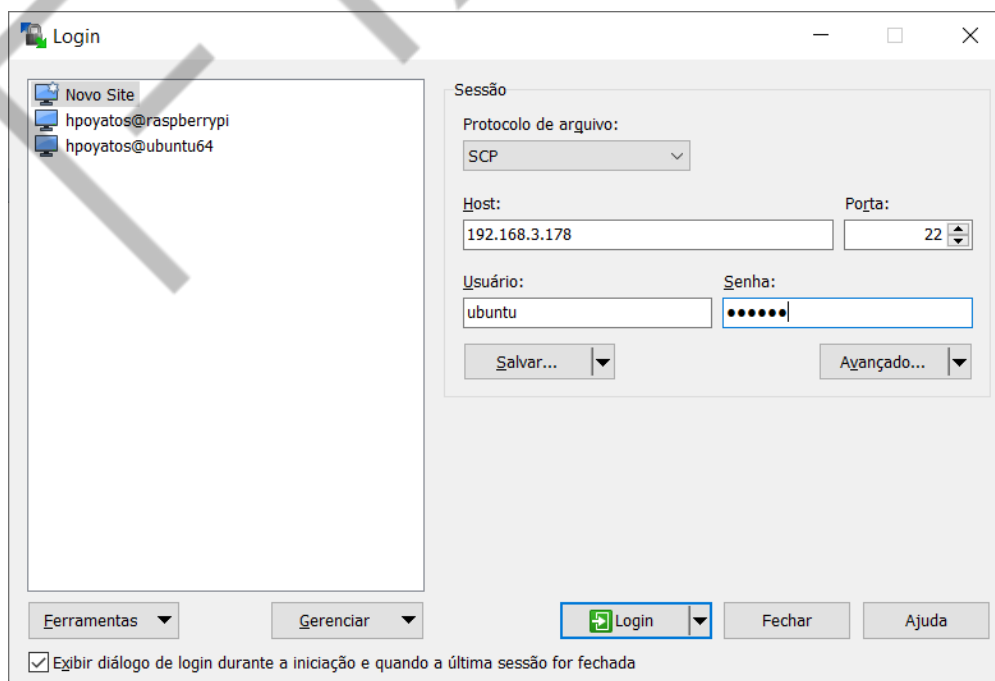


Figura 17 – Configurando o WinSCP para conexão SCP

Fonte: Elaborado pelo autor (2021)

Repare que a chave MD5 apresentada no WinSCP é rigorosamente a mesma que o cliente Putty recebeu (compare as duas figuras), e não poderia deixar de ser diferente. Essas chaves são apresentadas e a confirmação é solicitada apenas a primeira vez que os clientes as recebem, pois a partir da segunda conexão elas estão armazenadas em um arquivo de cache e são comparadas automaticamente, já que são consideradas conhecidas. O alerta só surge se a chave não coincidir mais, o que acontece quando o servidor muda (por alguma razão), nesse caso, é preciso atenção: o servidor mudou e você está ciente disso ou trata-se de um impostor?

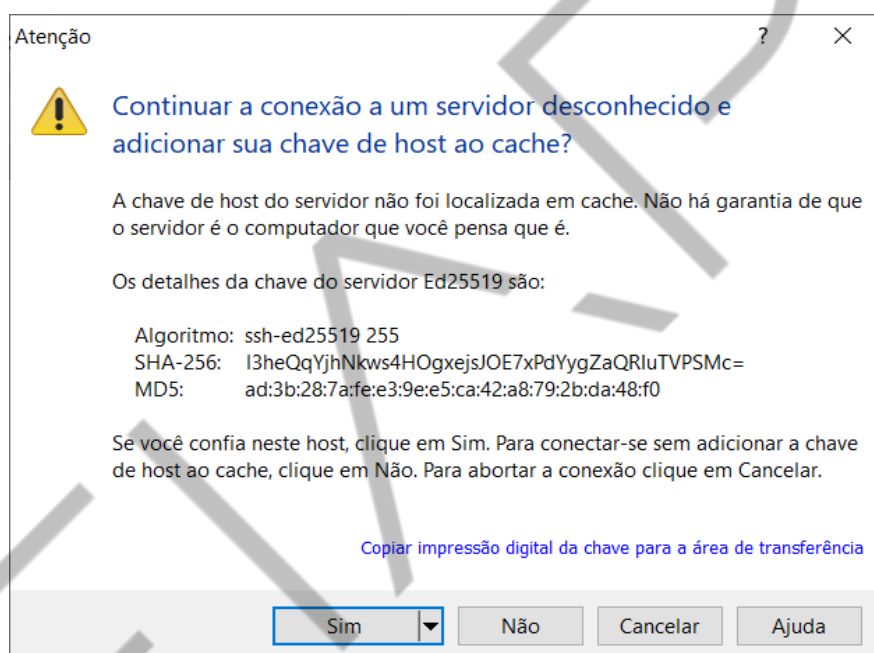


Figura 18 – Apresentação da chave WinSCP
Fonte: Elaborado pelo autor (2021)

Uma vez confirmada a chave, o cliente WinSCP apresenta o diretório home do usuário e permite a transferência de arquivos com o simples arrastar do mouse.

Muito prático!

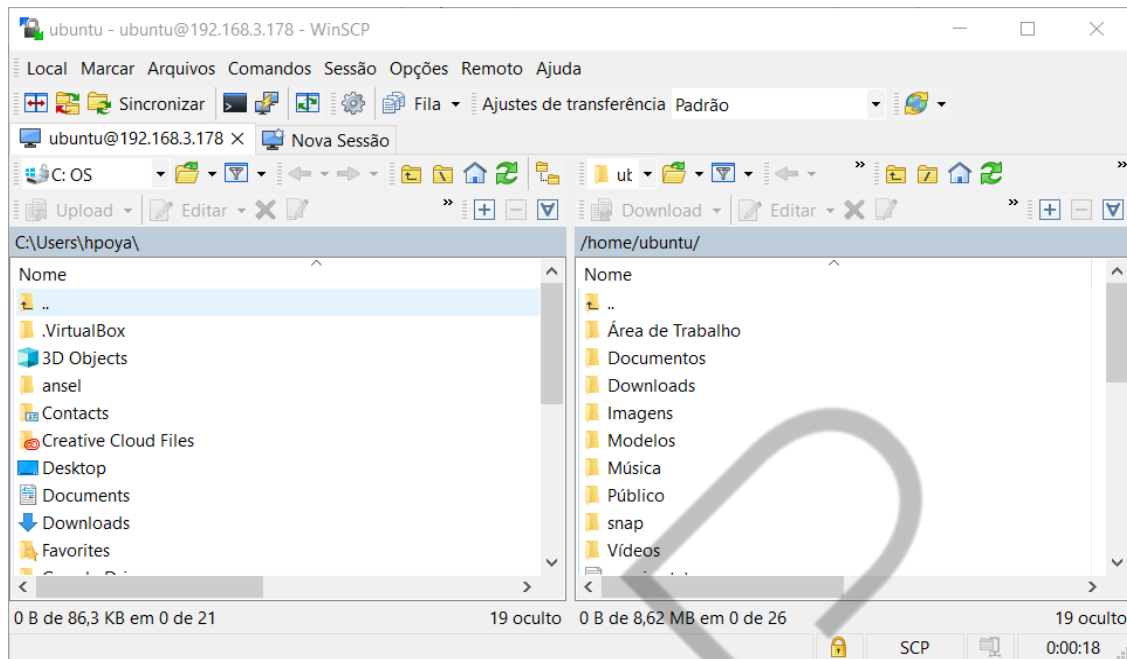


Figura 19 – Apresentação da chave WinSCP
Fonte: Elaborado pelo autor (2021)

Quer dizer que eu consigo transferir arquivos usando o serviço de SSH e o *Secure Copy* e não preciso do serviço de FTP? Bem, em tese, como visto aqui, não precisa, porém, como liberar *Secure Copy* significa na prática liberar algum tipo de terminal de comandos também, o serviço de FTP permanece adequado para vários casos de uso mais específicos, em que você basicamente quer liberar a transferência de arquivos e nada mais além disso.

3.3 Configurando o serviço OPENSSH-SERVER

Embora o serviço de SSH seja bem desenvolvido e seguro, sempre há espaço para que ele possa se tornar ainda mais seguro, afinal, por permitir o terminal remoto, que, com os privilégios certos, pode fazer absolutamente qualquer coisa no servidor, é um serviço também muito visado em ataques.

Vamos começar de alguns parâmetros que podem ser modificados no arquivo de configuração do serviço em `/etc/ssh/sshd_config`. Observe que este capítulo é apenas um recorte e que várias configurações podem ser realizadas.

Logo no começo do arquivo, temos a opção de trocar a porta padrão de serviço (que é a 22). Trocá-la pode ser uma estratégia para esconder o serviço ativo em um equipamento, já que ter a porta 22 aberta é “dar bandeira demais”.

```
#Port 22
#AddressFamily any
#ListenAddress 0.0.0.0
#ListenAddress ::
```

Código-fonte 7 – Descomente para restringir os usuários aos seus diretórios locais
Fonte: Elaborado pelo autor (2021)

Para modificá-la, basta descomentar a linha, removendo o símbolo “#” e trocando a numeração para algo de sua escolha (no exemplo aqui, 2222).

```
Port 2222
#AddressFamily any
#ListenAddress 0.0.0.0
#ListenAddress ::
```

Código-fonte 8 – Descomente para restringir os usuários aos seus diretórios locais
Fonte: Elaborado pelo autor (2021)

Observe também a instrução “ListenAddress”. Essa configuração é particularmente interessante e faz muito sentido em servidores que possuem diversas interfaces de rede. Vamos imaginar que existam duas interfaces de rede em um servidor, uma para uma rede interna e outra para uma rede externa (Internet). Por razões de segurança, desejo habilitar o serviço SSH apenas para a rede interna. Nesse caso, basta descomentar a linha “ListenAddress” e fornecer o endereçamento IPv4 ou IPv6 correspondente àquela interface de rede. *Voilà!* SSH aberto em uma rede, mas fechado em outra.

Qualquer modificação no arquivo de configuração exige não apenas salvá-lo, mas reiniciar o serviço usando o comando `systemctl`.

```
# Reiniciando o serviço openssh-server
$ sudo systemctl restart ssh
```

Comando de prompt 7 – Inicialização do openssh-server usando o comando `systemctl`
Fonte: Elaborado pelo autor (2021)

Muito bem, a partir de agora, para acessar o serviço de SSH, devo informar a porta 2222 no lugar a tradicional porta 22.

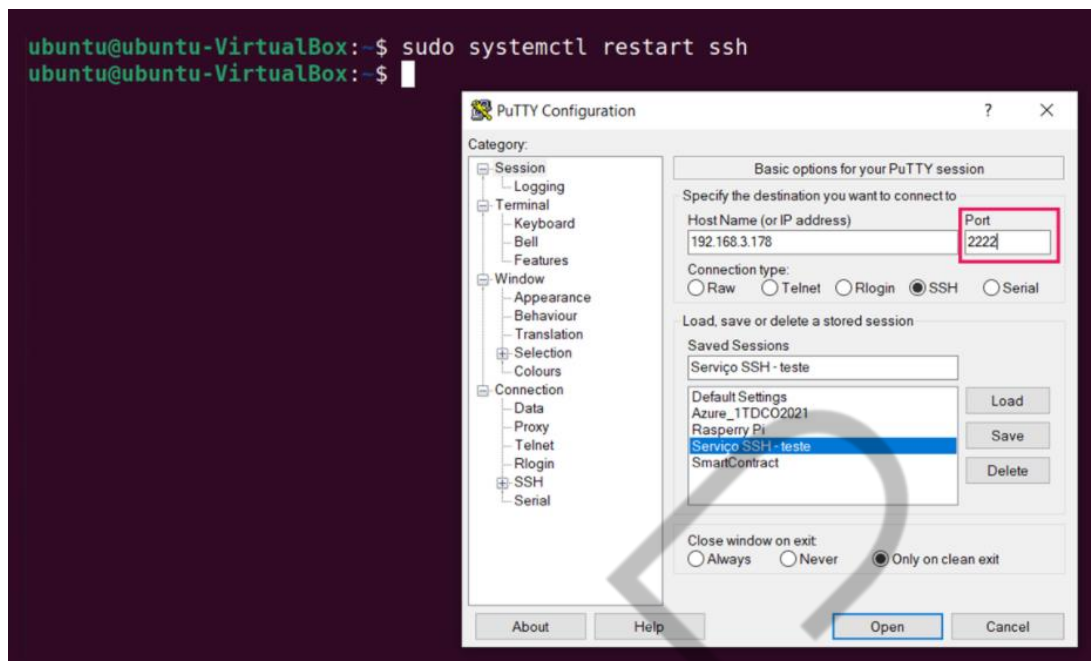


Figura 20 – Realizando o acesso SSH por meio da porta 2222

Fonte: Elaborado pelo autor (2021)

3.4 Realizando acesso ssh com uma chave no lugar da senha

Essa configuração certamente torna as coisas ainda mais seguras. Que tal substituir a senha do usuário (que pode ser um tanto ruim e facilmente quebrável) por uma chave de centenas de bits que demoraria centenas de anos para ser quebrada? A não ser que você tenha acesso a um computador quântico e queira gastar algumas dezenas de milhares de dólares para isso.

Para gerar a chave em questão é simples: se você quiser gerar em um MacOS ou Linux, basta usar o comando `ssh-keygen` e seguir alguns passos e moleza! Chave gerada! No Windows, podemos usar o Puttygen, que foi instalado com o Putty no passo anterior. Abra o Puttygen e peça para gerar a chave clicando em “Generate” (nesse exemplo, estou usando o padrão Ed25519, mas poderia ser RSA também).

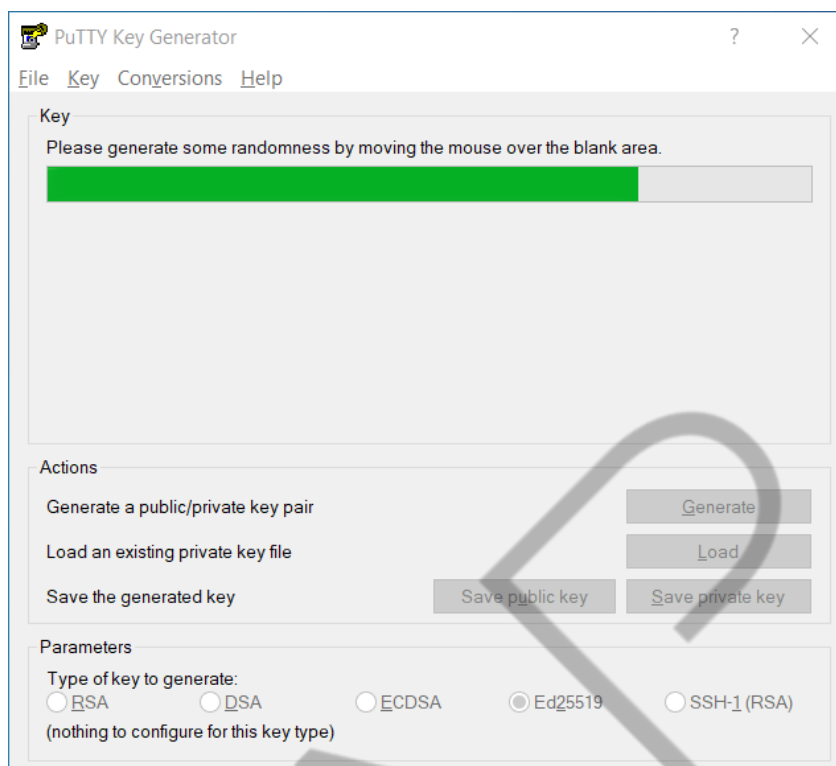


Figura 21 – Geração de chave usando o Puttygen
Fonte: Elaborado pelo autor (2021)

Depois de mexer o ponteiro do mouse aleatoriamente algumas vezes na área indicada, a chave é gerada.

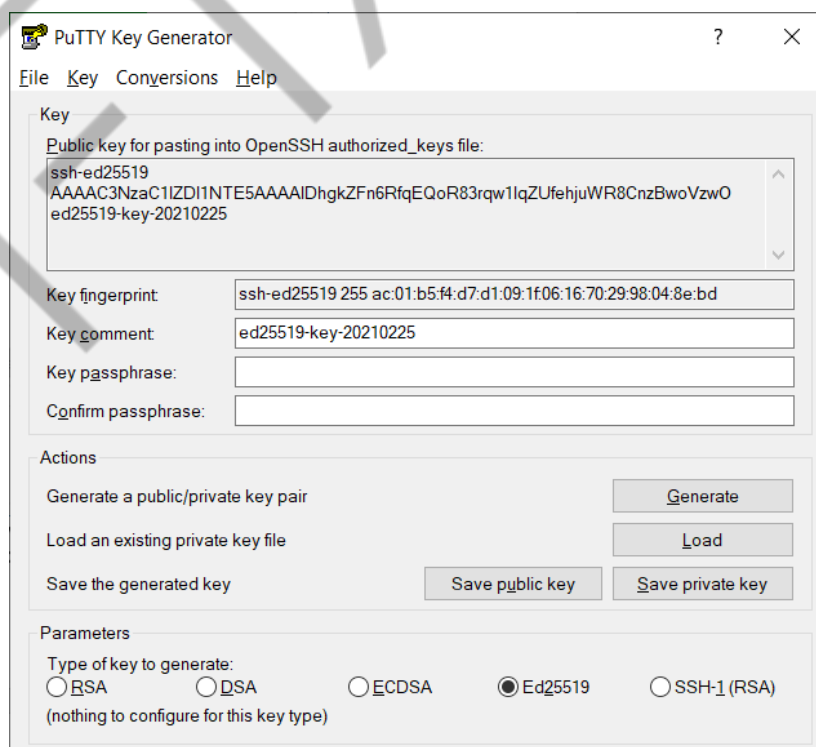


Figura 22 – Chave gerada pelo Puttygen
Fonte: Elaborado pelo autor (2021)

A chave pode ser copiada facilmente, como visto aqui.

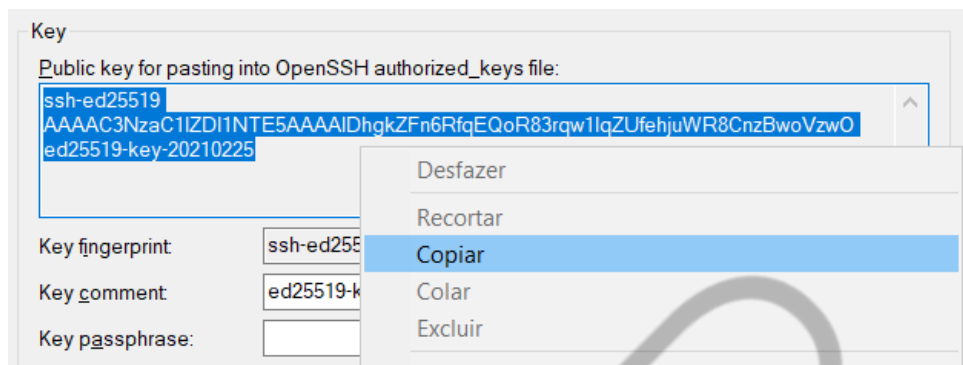


Figura 23 – Copiando a chave pública gerada no Puttygen
Fonte: Elaborado pelo autor (2021)

Feito isso, a chave deve ser copiada para dentro do arquivo `authorized_keys`, no diretório `.ssh` no diretório home do usuário que desejamos nos autenticar. Talvez o diretório `.ssh` não exista e cabe a você criá-lo com o comando `mkdir`. Provavelmente, o arquivo `authorized_keys` não exista (como era o caso aqui), mas assim que você usar o comando `echo` ou um editor de textos como o `vi` ou `nano` (sim, fique à vontade para abrir o editor e colar por lá!), o arquivo será criado.

```
ubuntu@ubuntu-VirtualBox:~$ cd .ssh/
ubuntu@ubuntu-VirtualBox:~/.ssh$ echo "ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIDhgkZF
n6RfqEQoR83rqw1IqZUfehjuWR8CnzBwoVzw0 ed25519-key-20210225" >> authorized_keys
ubuntu@ubuntu-VirtualBox:~/.ssh$ cat authorized_keys
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIDhgkZFn6RfqEQoR83rqw1IqZUfehjuWR8CnzBwoVzw0 e
d25519-key-20210225
ubuntu@ubuntu-VirtualBox:~/.ssh$
```

Figura 24 – Adicionando a chave pública no arquivo `/home/ubuntu/.ssh/authorized_keys`
Fonte: Elaborado pelo autor (2021)

A seguir, salve a chave privada em um arquivo `.ppk` clicando em “Save private key”. Caso tenha gerado no MacOS ou Linux com `ssh-keygen`, a extensão do arquivo de chave privada é diferente, mas o conteúdo é o mesmo.

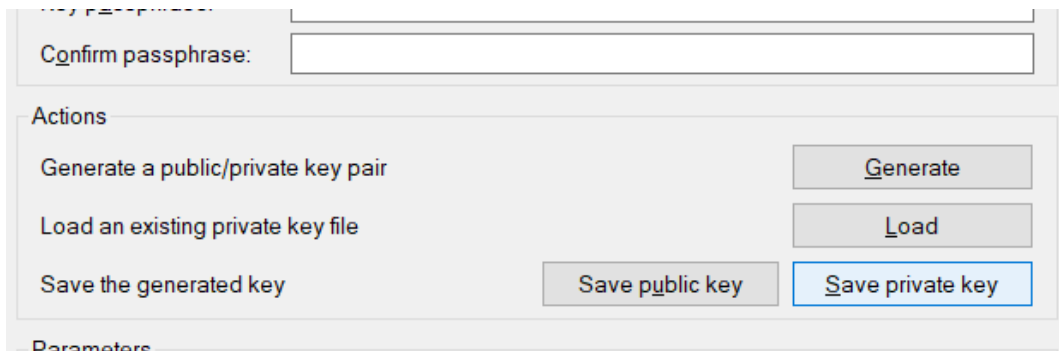


Figura 25 – Salvando o arquivo de chave privada do Puttygen
Fonte: Elaborado pelo autor (2021)

Voltemos agora ao arquivo de configuração `/etc/ssh/sshd_config`, você deve localizar este trecho em questão.

```
# To disable tunneled clear text passwords, change to no
here!
#PasswordAuthentication yes
#PermitEmptyPasswords no
```

Código-fonte 9 – Arquivo de configuração `/etc/ssh/sshd_config`
Fonte: Elaborado pelo autor (2021)

Sim, nós vamos proibir tentativas de acesso com senha a partir de agora, apenas conexões SSH com o arquivo de chave privada serão aceitas, o que tornará o serviço deveras mais seguro. Descomente a linha `PasswordAuthentication` e troque o “yes” para “no”.

```
# To disable tunneled clear text passwords, change to no
here!
PasswordAuthentication no
#PermitEmptyPasswords no
```

Código-fonte 10 – Arquivo de configuração `/etc/ssh/sshd_config` com `PasswordAuthentication` em no
Fonte: Elaborado pelo autor (2021)

Saia do arquivo de configuração salvando sua mudança e não deixe de rodar os comandos a seguir para definir as permissões corretas do diretório `.ssh` e arquivo `authorized_keys`. Substitua o usuário e grupo o comando `chown` pelo seu próprio usuário e grupo (no exemplo aqui, ambos chamam `ubuntu`). Se as permissões não estiverem corretas, acredite, o procedimento não funcionará!

```
# Definindo as permissões corretas  
$ chmod -R go= ~/.ssh  
$ chown -R ubuntu:ubuntu ~/.ssh
```

Comando de prompt 8 – Definindo as permissões de .ssh corretamente

Fonte: Elaborado pelo autor (2021)

Reinicie o serviço para que todas as modificações sejam contempladas!

```
# Reiniciando o serviço openssh-server  
$ sudo systemctl restart ssh
```

Comando de prompt 9 – Reinicialização do openssh-server usando o comando systemctl

Fonte: Elaborado pelo autor (2021)

A partir de agora, nada feito para acessar o serviço SSH: ao pedir o usuário, ele sequer solicita a senha e exibe uma mensagem de erro informando que a forma como a tentativa ocorreu é inválida.

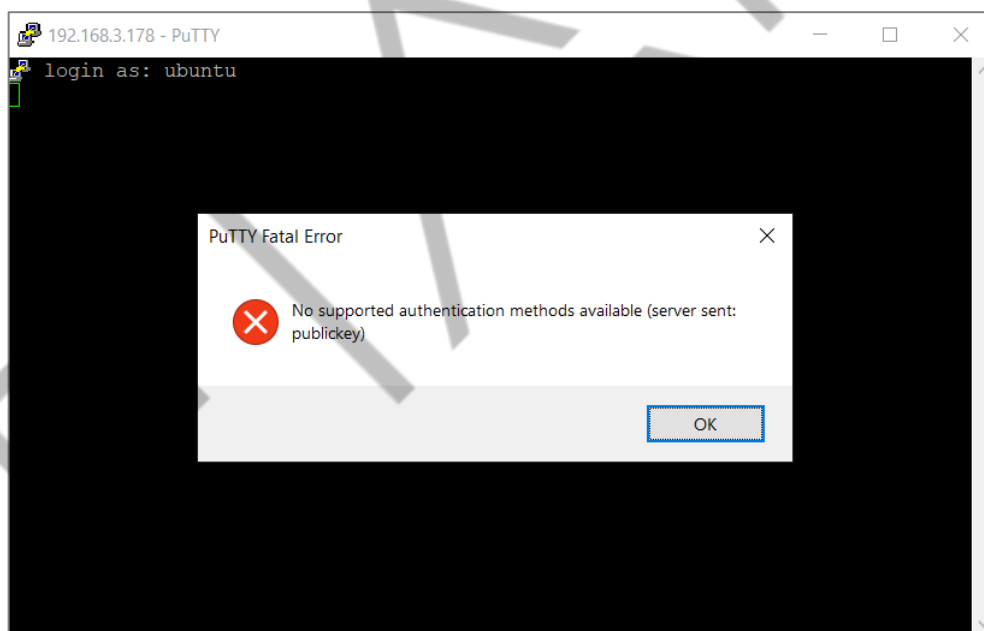


Figura 26 – Acesso proibido sem a chave ao usar o Putty

Fonte: Elaborado pelo autor (2021)

Para informar a chave privada no Putty, basta abri-lo e selecionar a sessão desejada. Mas calma, não clique em “Open” ainda!

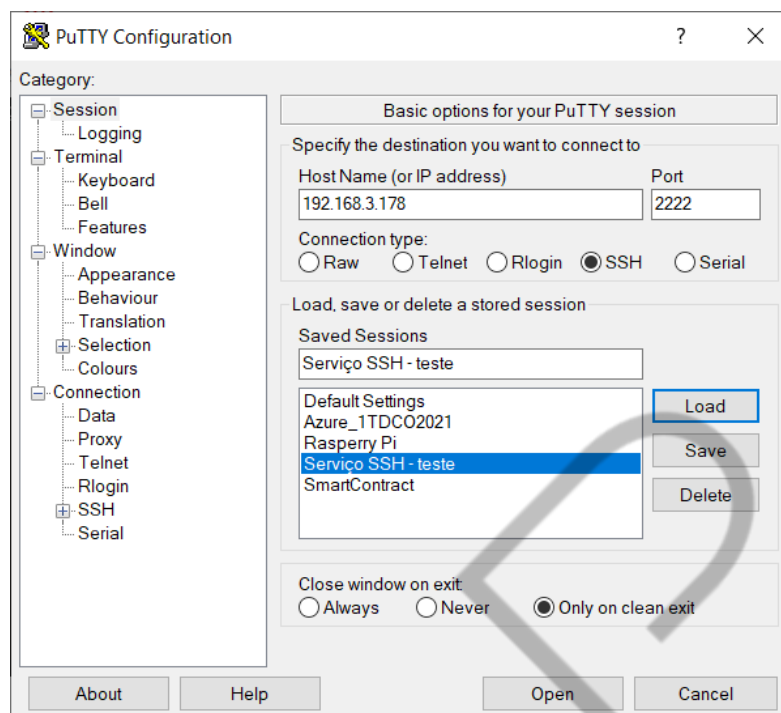


Figura 27 – Acesso proibido sem a chave ao usar o Putty
Fonte: Elaborado pelo autor (2021)

Na categoria *Connection > SSH > Auth*, clique no botão “Browse” e informe a chave privada do arquivo .ppk. Agora, sim, pode clicar em “Open”!

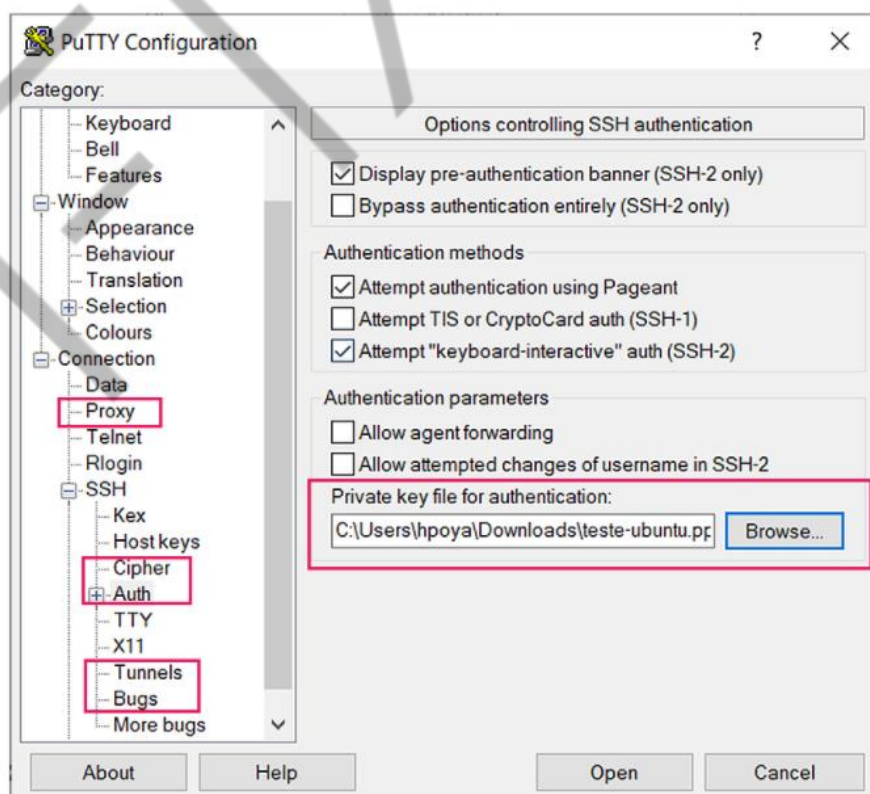
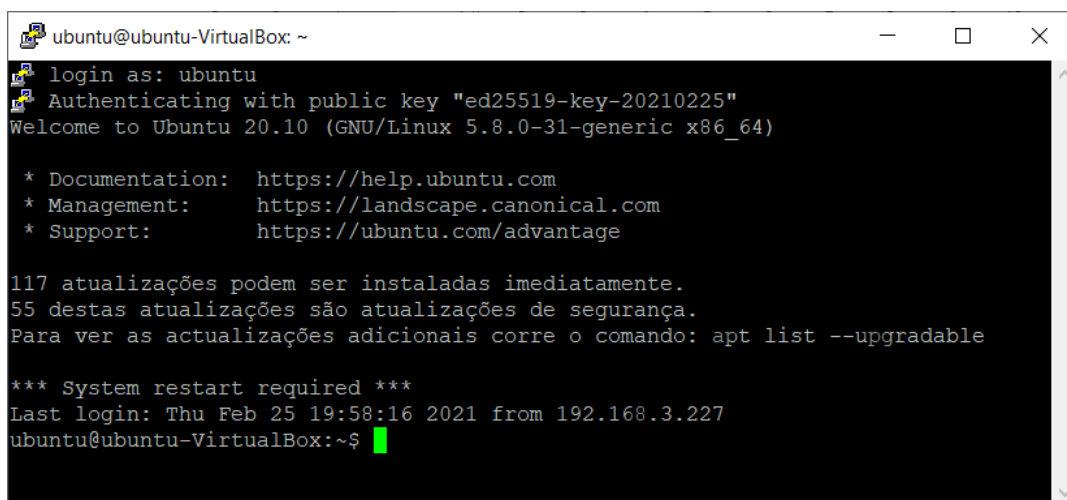


Figura 28 – Informando a chave privada arquivo .ppk no Putty
Fonte: Elaborado pelo autor (2021)

Uau!!! Funciona!!!

A terminal window titled 'ubuntu@ubuntu-VirtualBox: ~' showing the login process. It displays the prompt 'login as: ubuntu', followed by 'Authenticating with public key "ed25519-key-20210225"', and 'Welcome to Ubuntu 20.10 (GNU/Linux 5.8.0-31-generic x86_64)'. Below this, there are links for documentation, management, and support. Then, it shows the number of updates (117) and security updates (55), along with the command to list upgradable packages. It also indicates that a system restart is required and shows the last login time. The prompt 'ubuntu@ubuntu-VirtualBox:~\$' is visible at the bottom with a green cursor.

```
ubuntu@ubuntu-VirtualBox: ~
login as: ubuntu
Authenticating with public key "ed25519-key-20210225"
Welcome to Ubuntu 20.10 (GNU/Linux 5.8.0-31-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

117 atualizações podem ser instaladas imediatamente.
55 destas atualizações são atualizações de segurança.
Para ver as actualizações adicionais corre o comando: apt list --upgradable

*** System restart required ***
Last login: Thu Feb 25 19:58:16 2021 from 192.168.3.227
ubuntu@ubuntu-VirtualBox:~$
```

Figura 29 – Acesso com arquivo de chave privada usando o Putty
Fonte: Elaborado pelo autor (2021)

3.5 Port-knocking

As portas de serviço de um equipamento não precisam ficar 100% do tempo disponíveis e vulneráveis a ataques. Um recurso que pode aumentar muito a segurança é implementar o *port-knocking* (traduzido para “batidas na porta”) que remete àqueles filmes antigos em que a porta misteriosa seria aberta por alguém dentro do estabelecimento se o personagem batesse na porta na sequência esperada.

É exatamente isso que acontece, mas a porta aberta é uma porta de serviço liberada por uma regra de *firewall* disparada, se alguns pacotes de rede específicos atinjam algumas portas na sequência certa.

Para facilitar, vamos instalar o serviço chamado **knockd**, usando o comando `sudo apt install knockd` (Vide Figura “Instalar o serviço knockd”):


```
hpoyatos@fiap-virtualbox:~$ sudo apt install knockd
[sudo] password for hpoyatos:
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following NEW packages will be installed:
  knockd
0 upgraded, 1 newly installed, 0 to remove and 1 not upgraded.
Need to get 25.2 kB of archives.
After this operation, 91.1 kB of additional disk space will be used.
Get:1 http://nyc2.mirrors.digitalocean.com/debian stretch/main amd64 knockd amd6
4 0.7-1 [25.2 kB]
Fetched 25.2 kB in 0s (35.6 kB/s)
Selecting previously unselected package knockd.
(Reading database ... 32875 files and directories currently installed.)
Preparing to unpack .../knockd_0.7-1_amd64.deb ...
Unpacking knockd (0.7-1) ...
Progress: [ 66%] [#####.....]
Processing triggers for systemd (232-25+deb9u1) ...####.....]
hpoyatos@fiap-virtualbox:~$
```

Figura 30 – Instalar o serviço knockd
Fonte: Elaborado pelo autor (2018)

Para fazê-lo funcionar, vamos continuar com o exemplo de firewall que começamos no segundo capítulo desta Fase (Pinguim, Modo HARD! A coisa vai esquentar mais no sétimo capítulo, dedicado a *firewall*). Vamos realizar algumas mudanças de regras de firewall no arquivo `/etc/firewall.sh`. Repare a presença de uma regra baseada no parâmetro **-ctstate** que permite que conexões *entrantes* preexistentes continuem aceitas e não sejam derrubadas. Encerramos as regras com uma que rejeita todas as conexões entrantes além das estabelecidas (Vide Código-fonte “O arquivo `/etc/firewall.sh`”):


```
#!/bin/bash

# Limpa as regras de todas as correntes
iptables -F
iptables -X

# Estabelece as políticas padrão das correntes
iptables -P INPUT ACCEPT
iptables -P FORWARD ACCEPT
iptables -P OUTPUT ACCEPT

# Rejeita quaisquer conexões na porta 80
iptables -A INPUT -i lo -j ACCEPT

# Aceita e mantém conexões já estabelecidas
iptables -A INPUT -m conntrack --ctstate
RELATED,ESTABLISHED -j ACCEPT

# Aceita conexões na porta 80 livremente
iptables -A INPUT -p tcp -m tcp --dport 80 -j ACCEPT

# Rejeita outras conexões entrantes
iptables -A INPUT -j DROP
```

Código-fonte 11 – O arquivo /etc/firewall.sh
Fonte: Elaborado pelo autor (2018)

Ao confirmar as regras com o comando `sudo /etc/firewall.sh`, a conexão ssh atual não é derrubada devido à regra de aceite mencionada anteriormente.

O arquivo `/etc/knockd.conf` nos é entregue praticamente pronto, exigindo apenas uma alteração: a regra para abrir a porta 22 para meu IP deve ser criada com `iptables -I INPUT -s %IP% -p tcp --dport 22 -j ACCEPT` e não `iptables -A INPUT -s %IP% -p tcp --dport 22 -j ACCEPT`, a nova regra de firewall com o parâmetro “-A” ficaria após a regra de DROP e jamais seria alcançada. Sua correção como parâmetro “-I” para que a regra seja executada ANTES do DROP, resolvendo o problema (Vide Código-fonte “O arquivo /etc/knockd.conf com uma ligeira alteração”):

```
[options]
    UseSyslog

[openSSH]
    sequence      = 7000,8000,9000
    seq_timeout   = 5
    command       = /sbin/iptables -I INPUT -s %IP% -p
tcp --dport 22 -j ACCEPT
    tcpflags      = syn

[closeSSH]
    sequence      = 9000,8000,7000
    seq_timeout   = 5
    command       = /sbin/iptables -D INPUT -s %IP% -p
tcp --dport 22 -j ACCEPT
    tcpflags      = syn
```

Código-fonte 12 – O arquivo /etc/knockd.conf com uma ligeira alteração
Fonte: Elaborado pelo autor (2018)

Cabe uma última alteração no arquivo `/etc/default/knockd`, trocando o parâmetro **START_KNOCKD** de "0" para "1" (Vide Código-fonte "O arquivo `/etc/default/knock` alterado"):

```
# control if we start knockd at init or not
# 1 = start
# anything else = don't start
# PLEASE EDIT /etc/knockd.conf BEFORE ENABLING
START_KNOCKD=1

# command line options
#KNOCKD_OPTS="-i eth1"
```

Código-fonte 13 – O arquivo /etc/default/knock alterado
Fonte: Elaborado pelo autor (2018)

Basta agora subir o serviço **knockd** com o comando **sudo /etc/init.d/knockd start** e o *port-knocking* está devidamente implementado.

Cabe testar a segurança inicialmente realizando um escaneamento na porta 22 que indica uma filtragem ali; a tentativa de conexão direta na porta resulta em um *timeout* (Vide Figura "Verificar e testando acesso na porta 22 do ssh"):

```
hpoyatos@debian:~$ nmap 67.205.132.93 -p 22

Starting Nmap 7.40 ( https://nmap.org ) at 2018-04-16 22:18 -03
Nmap scan report for digitalocean (67.205.132.93)
Host is up (0.14s latency).
PORT      STATE      SERVICE
22/tcp    filtered  ssh

Nmap done: 1 IP address (1 host up) scanned in 1.55 seconds
hpoyatos@debian:~$
hpoyatos@debian:~$
hpoyatos@debian:~$ ssh hpoyatos@67.205.132.93
ssh: connect to host 67.205.132.93 port 22: Connection timed out
hpoyatos@debian:~$
```

Figura 31 – Verificando e testando acesso na porta 22 do ssh

Fonte: Elaborado pelo autor (2018)

Conforme visto no Código-fonte “O arquivo */etc/knockd.conf* com uma ligeira alteração”, o serviço espera por pacotes nas portas 7000, 8000 e 9000, respectivamente, e o próprio comando *nmap* pode ser usado para isso. Em Código-fonte “*Shell script que realiza o port-knocking*”, temos um pequeno *shell script* que dispara os pacotes nas portas 7000, 8000 e 9000, realizando o *port-knocking* e permitindo a conexão *ssh* na sequência. Caso queira realizar, troque o IP 67.205.135.93 pelo host em questão:

```
for x in 7000 8000 9000; do nmap -Pn --host_timeout 201 --
max-retries 0 -p $x 67.205.135.93; done
```

Código-fonte 14 – Shell script que realiza o *port-knocking*

Fonte: Elaborado pelo autor (2018)

```
hpoyatos@debian:~$ for x in 7000 8000 9000; do nmap -Pn --host_timeout 201 --max-retries 0 -p $x 67.205.132.93; done

Starting Nmap 7.40 ( https://nmap.org ) at 2018-04-16 23:00 -03
Warning: 67.205.132.93 giving up on port because retransmission cap hit (0).
Nmap scan report for digitalocean (67.205.132.93)
Host is up.
PORT      STATE      SERVICE
7000/tcp  filtered  afs3-fileserver

Nmap done: 1 IP address (1 host up) scanned in 1.06 seconds

Starting Nmap 7.40 ( https://nmap.org ) at 2018-04-16 23:00 -03
Warning: 67.205.132.93 giving up on port because retransmission cap hit (0).
Nmap scan report for digitalocean (67.205.132.93)
Host is up.
PORT      STATE      SERVICE
8000/tcp  filtered  http-alt

Nmap done: 1 IP address (1 host up) scanned in 1.10 seconds

Starting Nmap 7.40 ( https://nmap.org ) at 2018-04-16 23:00 -03
Warning: 67.205.132.93 giving up on port because retransmission cap hit (0).
Nmap scan report for digitalocean (67.205.132.93)
Host is up.
PORT      STATE      SERVICE
9000/tcp  filtered  cslistener

Nmap done: 1 IP address (1 host up) scanned in 1.11 seconds
hpoyatos@debian:~$ ssh hpoyatos@67.205.132.93
hpoyatos@67.205.132.93's password: █
```

Figura 32 – *Port-knocking* realizado com sucesso

Fonte: Elaborado pelo autor (2018)

Conforme pode ser visto em Código-fonte “O arquivo `/etc/knockd.conf` com uma ligeira alteração”, o serviço `knockd` pode fechar a porta se os pacotes forem disparados na sequência contrária (portas 9000, 8000 e 7000), cabendo uma ligeira modificação em Código-fonte “Shell script que realiza o *port-knocking*” para tal.

Para aumentar a segurança, altere todas as portas do exemplo mostrado aqui. Comece alterando o arquivo `/etc/ssh/sshd_config` para que o serviço `ssh` suba em uma porta diferente da 22. Depois, altere `/etc/knockd.conf` para que a regra do firewall libere a mesma porta que o `ssh` subirá. Além disso, modifique as portas que abrirão e fecharão o `ssh`; o *port-knocking* deve ocorrer em portas fora do padrão, pois as primeiras que um atacante tentará bater são justamente 7000, 8000 e 9000, pois são o padrão predefinido por `knockd`. Suba os serviços de `sshd` e `knockd` e sintá-se mais seguro do que antes.

REFERÊNCIA

BOUCHERON, B. **Como configurar as chaves SSH no Ubuntu 20.04**. 2020.
Disponível em: <<https://www.digitalocean.com/community/tutorials/how-to-set-up-ssh-keys-on-ubuntu-20-04-pt>>. Acesso em: 4 mar. 2021.

EMEND